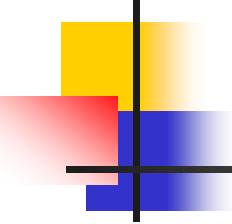


웹 프로그래밍

제6장 JSP

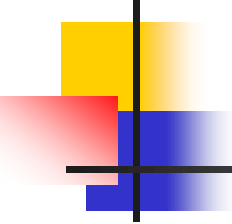
생능출판사



6. JSP

교재 p.543

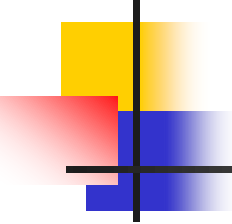
- 6.1 JSP 기초
- 6.2 JSP 기본 요소
- 6.3 JSP 기본 문법
- 6.4 JSP 내장 객체
- 6.5 JSP와 데이터베이스 연동
- 6.6 JSP 프로그래밍 예제



6.1 JSP 기초

교재 p.545

- JSP 개요
- JSP 처리 방식



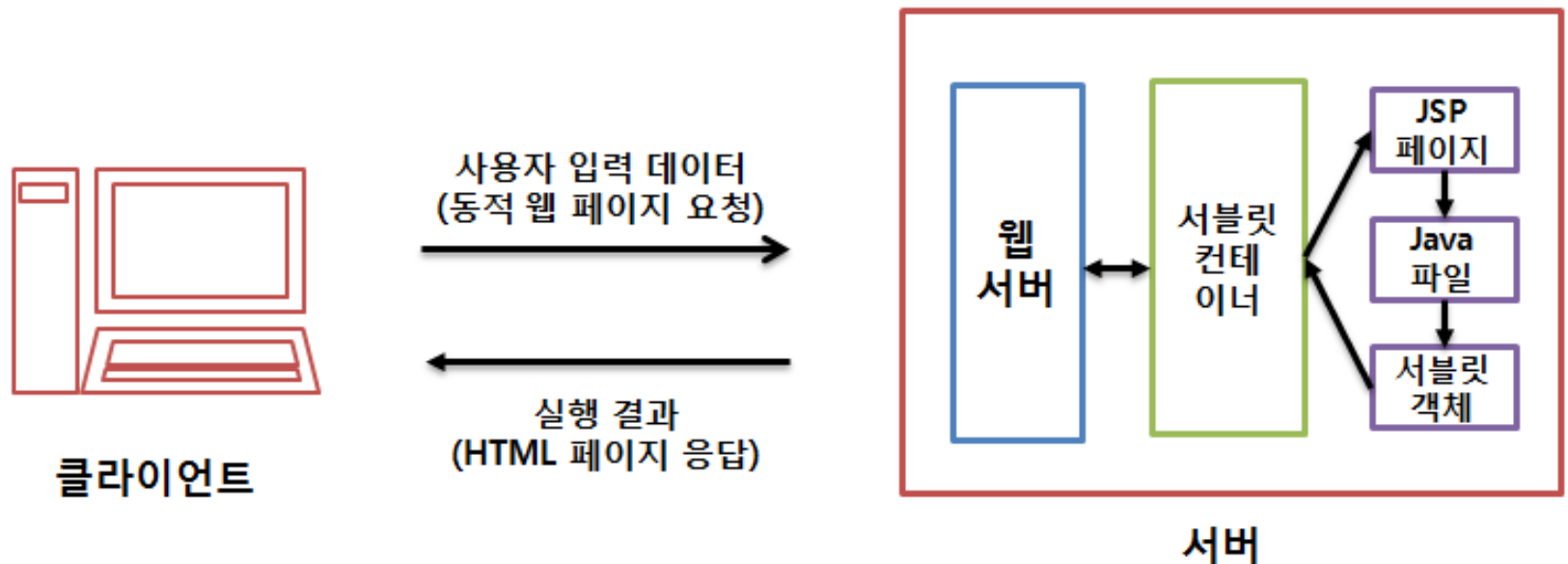
6.1.1 JSP 개요

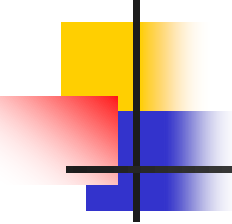
교재 p.545

- 자바를 기반으로 하는 동적 웹 사이트 구축 언어
- 서버 사이드 스크립트(server side script) 언어
- JSP 특징
 - 플랫폼에 독립적
 - 서버 자원의 효율적 관리
 - 컴포넌트 기반 개발
 - 비즈니스 로직과 프리젠테이션 로직의 분리

6.1.2 JSP 처리 방식

교재 p.547

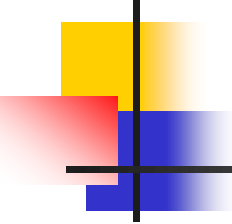




6.2 JSP 기본 요소

교재 p.548

- 지시문
- 스크립팅 요소
- 액션 태그
- 내장 객체
- 간단한 예제 작성



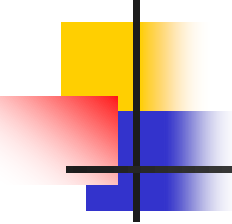
6.2.1 지시문

교재 p.548

- 페이지의 처리와 관련된 정보 기술

형 식 : <%@ 지시문_이름 속성_정의_리스트 %>

속성_정의 : 속성 = "속성_값"

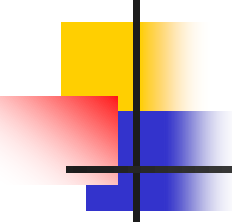


6.2.1 지시문

교재 p.548

- page 지시문

사용 예 : <%@ page contentType="text/html; charset=euc-kr" %>

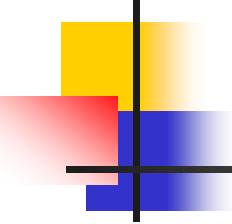


6.2.1 지시문

교재 p.549

[표 6.1] page 지시문의 속성 및 기본값

속성	설명
contentType	MIME 타입과 문자 셋
import	임포트할 패키지
pageEncoding	서블릿으로 변환할 때의 문자 인코딩 방식
session	HTTP 세션 사용 여부
buffer	클라이언트로 전송되는 out 객체의 버퍼 크기
autoFlush	버퍼가 찼을 때 클라이언트로 자동 출력 여부
extends	현재 페이지가 상속받을 수퍼 클래스
errorpage	에러를 출력할 페이지의 url
isErrorpage	현재 페이지를 에러 페이지로 사용 여부
isELIgnored	식 언어(expression language) 사용 여부
isThreadSafe	다중 스레드 사용 여부
info	JSP 페이지에 대한 정보
language	JSP 페이지에서 사용하는 언어

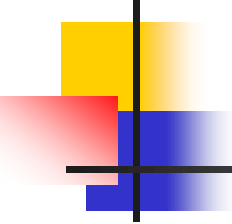


6.2.1 지시문

교재 p.550

- include 지시문

형 식 : <%@ include file="파일의 url" %>



6.2.1 지시문

교재 p.550

- 예제 : include-directive.jsp

```
4 : <body>
5 :     include 지시문 사용하기
6 :     <br><hr>
7 :     <%@ include file="include-test.jsp" %>
8 : </body>
```

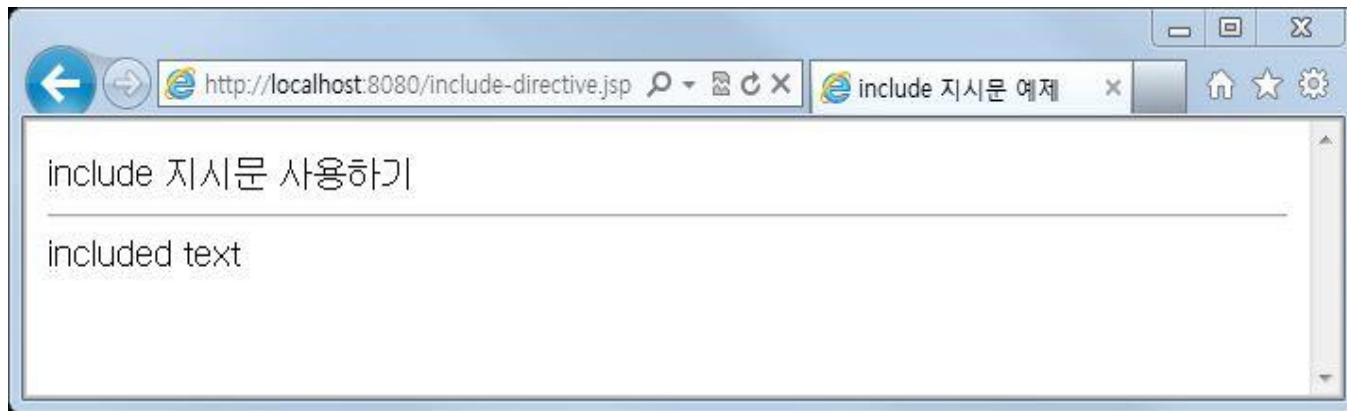
- 예제 : include-test.jsp

```
4 : <body>
5 :     included text
6 : </body>
```

6.2.1 지시문

교재 p.551

■ 실행 결과



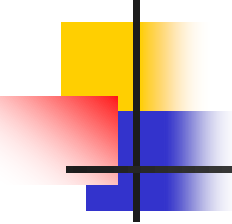


교재 p.551

- taglib 지시문

```
형 식 : <%@ taglib uri = "태그 라이브러리의 uri"
        prefix = "사용자 정의 태그의 접두어" %>
```

사용 예 : <%@ taglib uri="userLib" prefix="userTag" %>



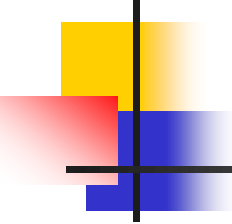
6.2.2 스크립팅 요소

교재 p.552

■ 선언문

```
형 식 (변 수) : <%!  
                자료형 변수_리스트  
                %>
```

```
사용 예 : <%!  
          var a;  
          var b, c;  
          var d = 1;  
          %>
```



6.2.2 스크립팅 요소

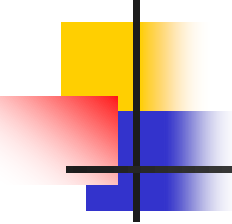
교재 p.553

■ 스크립트릿

형 식 : <%
자바_코드
%>

사용 예 : <%
int x = 2;
int y = 3;
int z;

z = add(x, y);
%>



6.2.2 스크립팅 요소

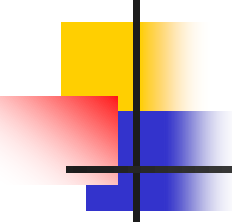
교재 p.554

■ 식

형 식 : $\langle \% = \text{자바_식} \% \rangle$

사용 예 (변수) : $\langle \% = x \% \rangle + \langle \% = y \% \rangle = \langle \% = z \% \rangle$

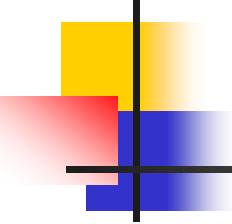
사용 예 (수식) : $\langle \% = x \% \rangle + \langle \% = y \% \rangle = \langle \% = x+y \% \rangle$



6.2.3 액션 태그

교재 p.554

- 어떠한 액션이 발생하는 시점에 실행
- XML을 기반으로 하기 때문에 시작 태그가 있으면 반드시 종료 태그가 있어야 함
- 태그는 접두어 'jsp:'로 시작하여야 함

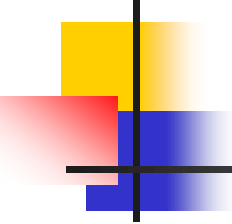


6.2.3 액션 태그

교재 p.555

- <jsp:include> 태그

사용 예 : <jsp:include page="a.jsp" flush="false"/>

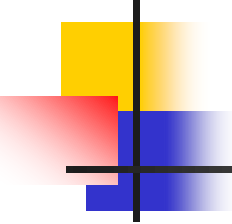


6.2.3 액션 태그

교재 p.556

- <jsp:forward> 태그

사용 예 : <jsp:forward page="a.jsp"/>

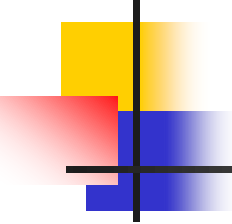


6.2.3 액션 태그

교재 p.556

- <jsp:param> 태그

```
사용 예 : <jsp:forward page="a.jsp">  
          <jsp:param name="param_name" value="value1"/>  
        </jsp:forward>
```

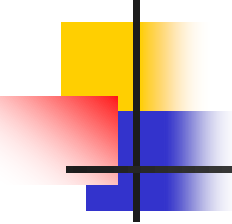


6.2.3 액션 태그

교재 p.556

- <jsp:plug-in> 태그

사용 예 : <jsp:plug-in type="applet" code="applet_exe.class"
codebase="/class_code"/>



6.2.3 액션 태그

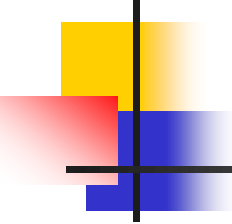
교재 p.557

- `<jsp:useBean>` 태그

사용 예 : `<jsp:useBean id="useBean" class="Bean.class" scope="page"/>`

- `<jsp:setProperty>` 태그

사용 예 : `<jsp:useBean id="bean">
 <jsp:setProperty name="bean" property="title"
 value="web programming"/>
</jsp:useBean>`



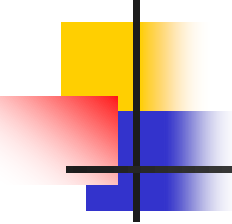
6.2.3 액션 태그

교재 p.557

■ <jsp:getProperty> 태그

사용 예 :

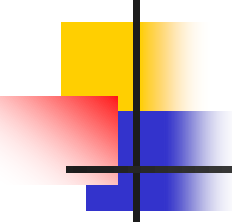
```
<jsp:useBean id="bean">  
  <jsp:setProperty name="bean" property="title"  
                    value="web programming"/>  
  책 이름은 <jsp:getProperty name="bean" property="title"/>  
  입니다.  
</jsp:useBean>
```



6.2.4 내장 객체

교재 p.558

- 정의할 필요없이 자동으로 제공되는 객체
- `<% ... %>` 또는 `<%= ... %>` 내에서 사용

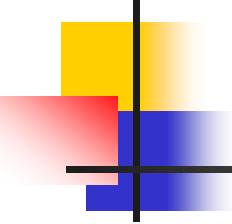


6.2.4 내장 객체

교재 p.558

[표 6.4] JSP 내장 객체 및 기능

내장 객체	설명
request	클라이언트의 요청 처리
response	클라이언트의 요청에 대한 응답
out	출력 스트림 처리
session	클라이언트의 세션 정보 처리
application	애플리케이션 정보 처리
pageContext	JSP 페이지의 내용(context)
config	JSP 페이지의 초기화(initialization) 매개변수 저장
page	현재의 JSP 페이지
exception	예외 처리



6.2.5 간단한 예제 작성

교재 p.559

■ 예제 : welcome.jsp

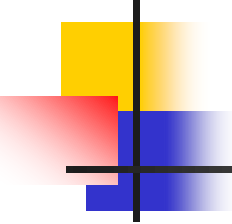
```
6 :      <%  
7 :          out.println("Welcome to JSP !");  
8 :      %>  
9 :      <br> <hr>  
10 :      <%  
11 :          String text = "Welcome to JSP !";  
12 :      %>  
13 :      <%= text%>
```

6.2.5 간단한 예제 작성

교재 p.560

- 실행 결과

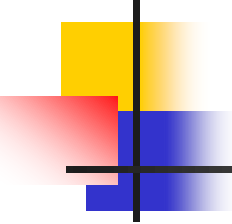




6.3 JSP 기본 문법

교재 p.560

- 기본 태그
- 자료형 및 변수
- 연산자
- 문장
- 클래스
- 함수
- 예외 처리

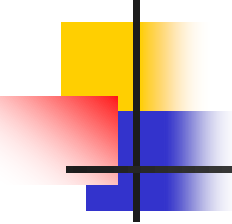


6.3.1 기본 태그

교재 p.560

- 식 태그

형 식 : <%= 자바_식 %>

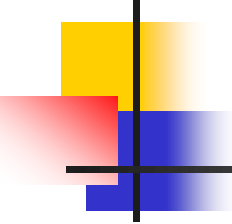


6.3.1 기본 태그

교재 p.561

- 스크립트릿 태그

형 식 : <% 자바_코드 %>



6.3.1 기본 태그

교재 p.561

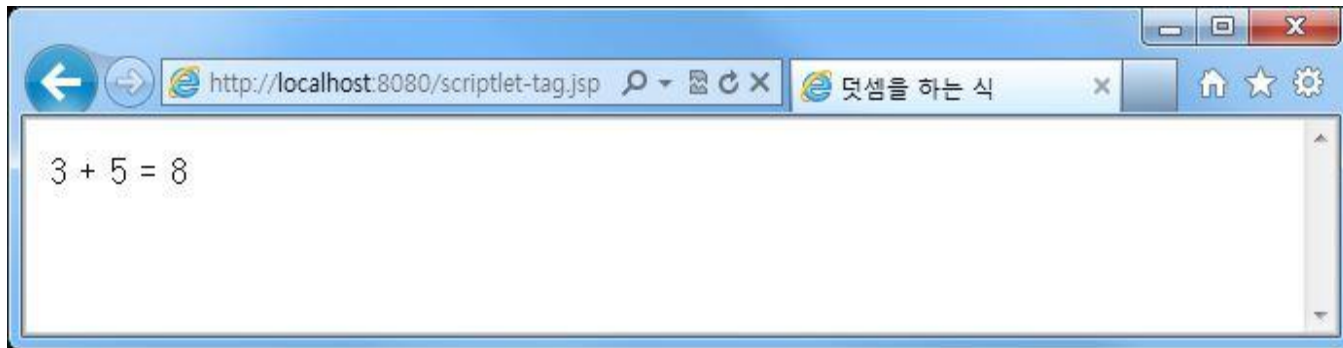
■ 예제 : scriptlet-tag.jsp

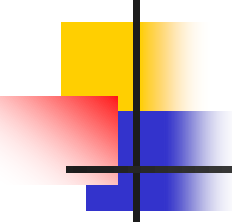
```
5 :   <%  
6 :  
7 :       int a = 3;  
8 :       int b = 5;  
9 :   %>  
10 :  
11 :   3 + 5 = <%= a + b %>
```

6.3.1 기본 태그

교재 p.561

- 실행 결과





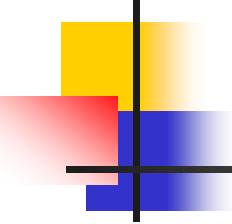
6.3.1 기본 태그

교재 p.562

■ 주석

[표 6.6] JSP 주석의 종류

형식	설명
<!-- 주석 -->	HTML 주석
<%-- 주석 --%>	원본 소스에서만 주석이 보이고 클라이언트의 소스 보기에서는 보이지 않으므로, 보안이 필요한 경우에 활용
// 주석	한 줄 이내의 주석
/* 주석 */	여러 줄의 주석



6.3.1 기본 태그

교재 p.562

■ 예제 : comment.jsp

```
5 : <!-- HTML 주석입니다.-->
6 : <%-- 소스 보기에서는 주석이 안 보입니다.
7 :       이것은 블록 단위 주석입니다.
8 :     --%>
9 : <%
10 :       out.println("주석 예제입니다.");
11 :     %>
```

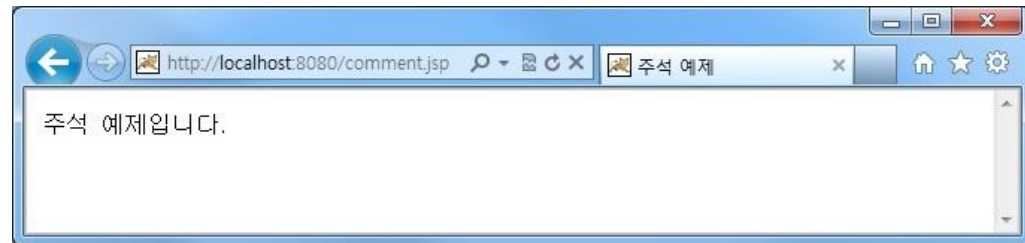
6.3.1 기본 태그

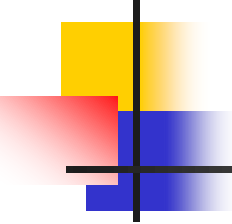
교재 p.563

■ 실행 결과



```
1
2 <html>
3     <head>
4         <title> 주석 예제 </title>
5     </head>
6     <body>
7         <!-- HTML 주석입니다.-->
8
9         주석 예제입니다.
10
11     </body>
12 </html>
```





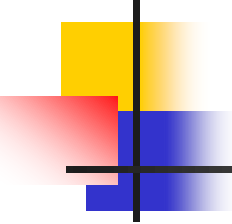
6.3.2 자료형 및 변수

교재 p.564

■ 변수

[표 6.7] JSP의 기본 자료형

자료형	설명
boolean	true 또는 false(1 bit)
char	자바의 유니코드 문자(2 bytes)
byte	-128 ~ 127 범위의 정수(1 byte)
short	-32,768 ~ 32,767 범위의 정수(2 bytes)
int	-2,147,483,648 ~ 2,147,483,647 범위의 정수(4 bytes)
long	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807 범위의 정수(8 bytes)
float	부동소수점 실수(4 bytes)
double	부동소수점 실수(8 bytes)



6.3.2 자료형 및 변수

교재 p.564

- 변수(계속)
 - 변수 선언

형 식 :

자료형 변수_선언_리스트;

변수_선언 : 변수_이름[= 값]

- 선언 예

사용 예 : int a;
float b, c;
int d = 1;

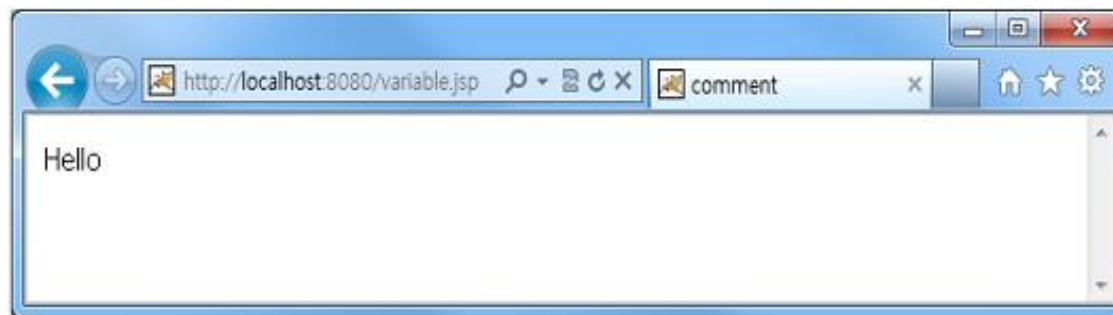
6.3.2 자료형 및 변수

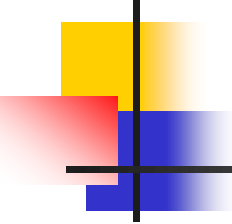
교재 p.565

■ 예제 : variable.jsp

```
1 : <%  
2 :     String text;  
3 :     text = "Hello";  
4 : %>  
5 : <%= text %>
```

■ 실행 결과



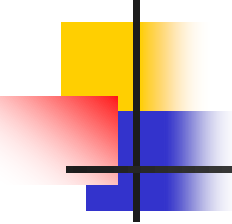


6.3.2 자료형 및 변수

교재 p.565

- 배열

- 하나의 변수에 같은 자료형의 하나 이상의 값을 저장하고자 할 때 사용



6.3.2 자료형 및 변수

교재 p.566

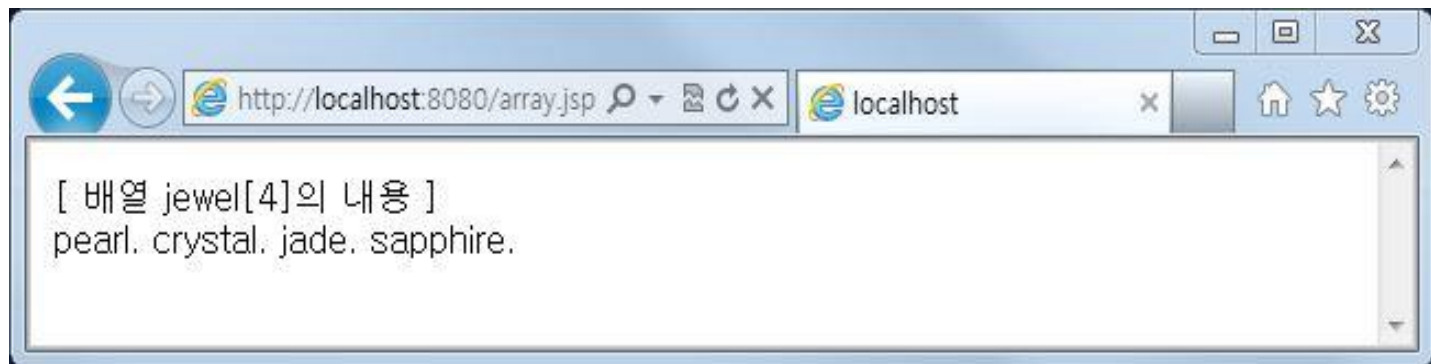
■ 예제 : array.jsp

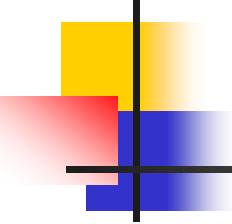
```
2 : <%  
3 :     String[] jewel = new String[4];  
4 :  
5 :     jewel[0] = "pearl";  
6 :     jewel[1] = "crystal";  
7 :     jewel[2] = "jade";  
8 :     jewel[3] = "sapphire";  
9 : %>  
10 : [ 배열 jewel[4]의 내용 ]<br>  
11 : <%= jewel[0]%>. <%= jewel[1]%>. <%= jewel[2]%>.  
    <%= jewel[3]%>.
```


6.3.2 자료형 및 변수

교재 p.566

■ 실행 결과



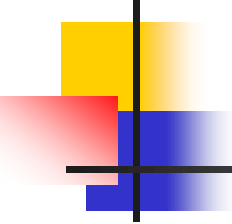


6.3.3 연산자

교재 p.566

- 산술 연산자
 - 산술 연산

사용 예 :
 $a + b$
 $a - b$
 $a * b$
 a / b
 $a \% b$

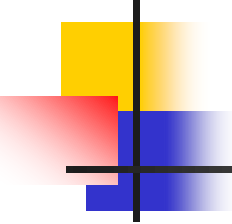


6.3.3 연산자

교재 p.567

- 증감 연산자
 - ++, -- 기호를 변수 앞이나 뒤에 붙임
 - 변수 값을 1씩 증가 또는 감소

사용 예 : ++a
 a++
 --a
 a--



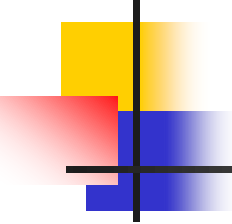
6.3.3 연산자

교재 p.568

- 관계 연산자
 - 값의 대소 관계를 판단
 - 참(true) 또는 거짓(false)을 구함

사용 예 :

- $a > b$
- $a \geq b$
- $a < b$
- $a \leq b$
- $a == b$
- $a != b$



6.3.3 연산자

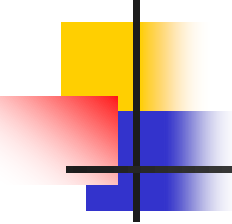
교재 p.569

- 논리 연산자
 - 참(true) 또는 거짓(false)을 구함

사용 예 :

```
a && b
a || b
!a

a > b && c == d
a > 7 || b <= 100
```



6.3.3 연산자

교재 p.569

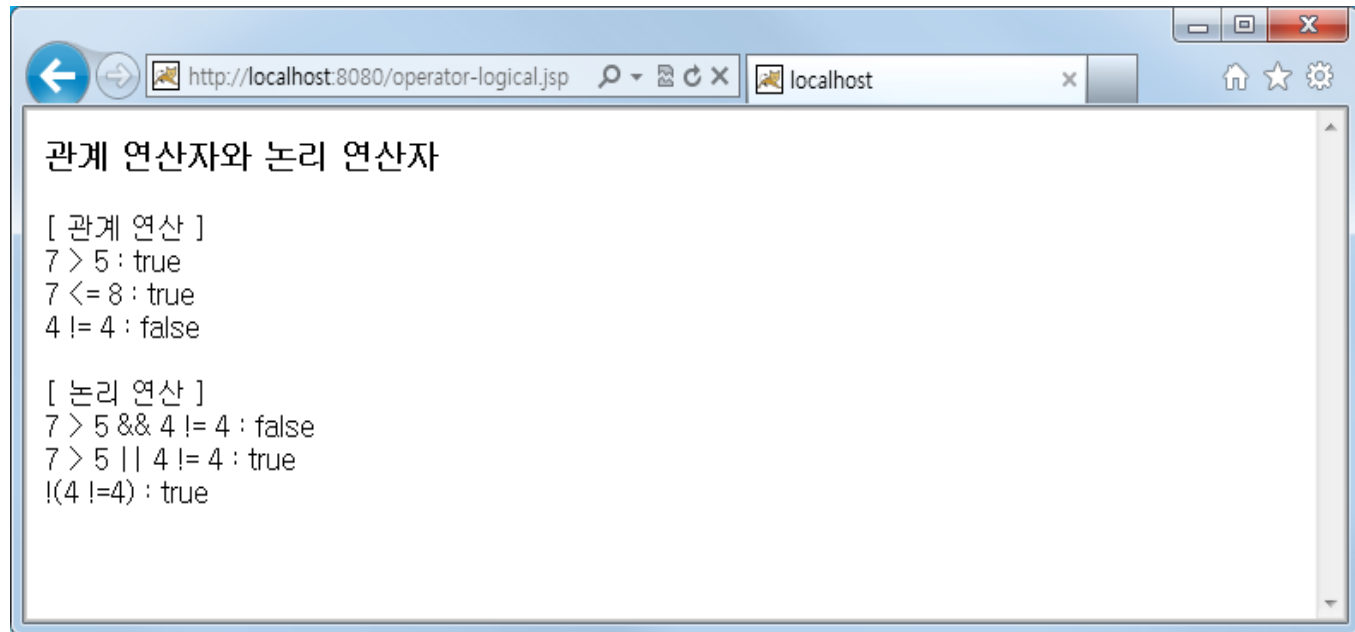
■ 예제 : operator-logical.jsp

```
2 : <h3> 관계 연산자와 논리 연산자 </h3>
3 :
4 : [ 관계 연산 ] <br>
5 : 7 > 5 : <%= 7 > 5 %> <br>
6 : 7 <= 8 : <%= 7 <= 8 %> <br>
7 : 4 != 4 : <%= 4 != 4 %> <br>
8 :
9 : <br> [ 논리 연산 ] <br>
10 : 7 > 5 && 4 != 4 : <%= 7 > 5 && 4 != 4 %> <br>
11 : 7 > 5 || 4 != 4 : <%= 7 > 5 || 4 != 4 %> <br>
12 : !(4 !=4) : <%= !(4 != 4) %>
```

6.3.3 연산자

교재 p.569

■ 실행 결과

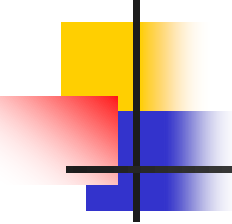


A screenshot of a web browser window. The address bar shows 'http://localhost:8080/operator-logical.jsp'. The page content is as follows:

```
관계 연산자와 논리 연산자

[ 관계 연산 ]
7 > 5 : true
7 <= 8 : true
4 != 4 : false

[ 논리 연산 ]
7 > 5 && 4 != 4 : false
7 > 5 || 4 != 4 : true
!(4 != 4) : true
```



6.3.3 연산자

교재 p.570

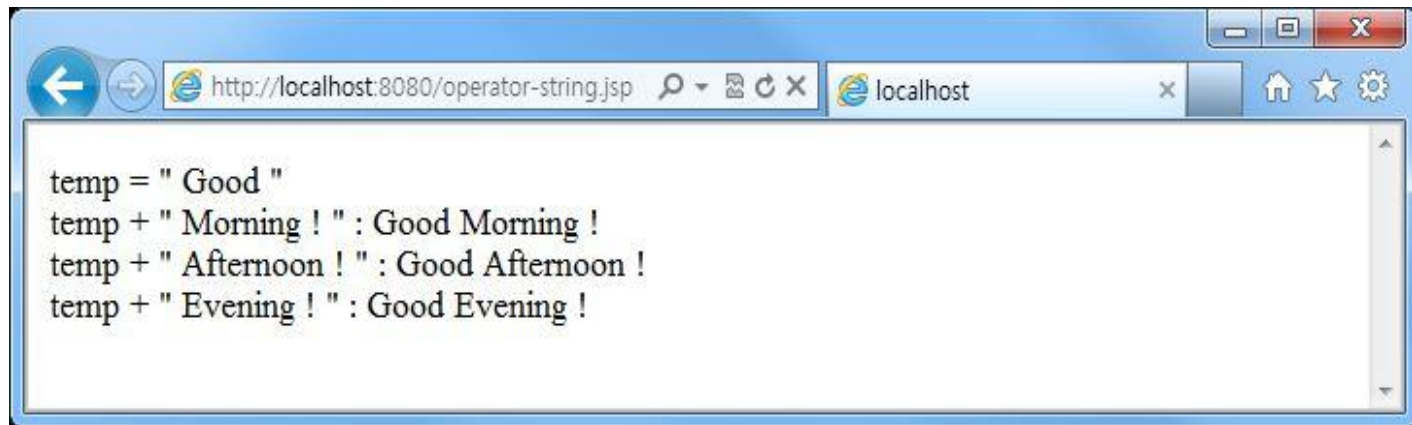
- 문자열 연산자
 - + 연산자 사용
- 예제 : operator-string.jsp

```
1 : <%  
2 :     String temp;  
3 :     temp = " Good ";  
4 : %>  
5 : temp = " Good " <br>  
6 : temp + " Morning ! " : <%= temp + " Morning ! " %> <br>  
7 : temp + " Afternoon ! " : <%= temp + " Afternoon ! " %> <br>  
8 : temp + " Evening ! " : <%= temp + " Evening ! " %>
```


6.3.3 연산자

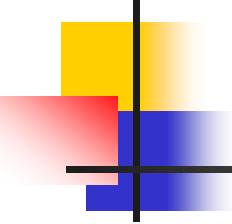
교재 p.570

■ 실행 결과



A screenshot of a web browser window. The address bar shows the URL `http://localhost:8080/operator-string.jsp`. The page content displays the following text:

```
temp = " Good "  
temp + " Morning ! " : Good Morning !  
temp + " Afternoon ! " : Good Afternoon !  
temp + " Evening ! " : Good Evening !
```



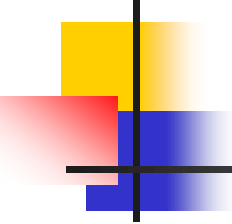
6.3.3 연산자

교재 p.570

- 배정 연산자
 - 변수에 값 저장
 - 할당 연산자 또는 대입 연산자

사용 예 :

```
a = 1  
a = b  
a += b  
a -= b  
a *= b  
a /= b  
a %= b
```



6.3.3 연산자

교재 p.571

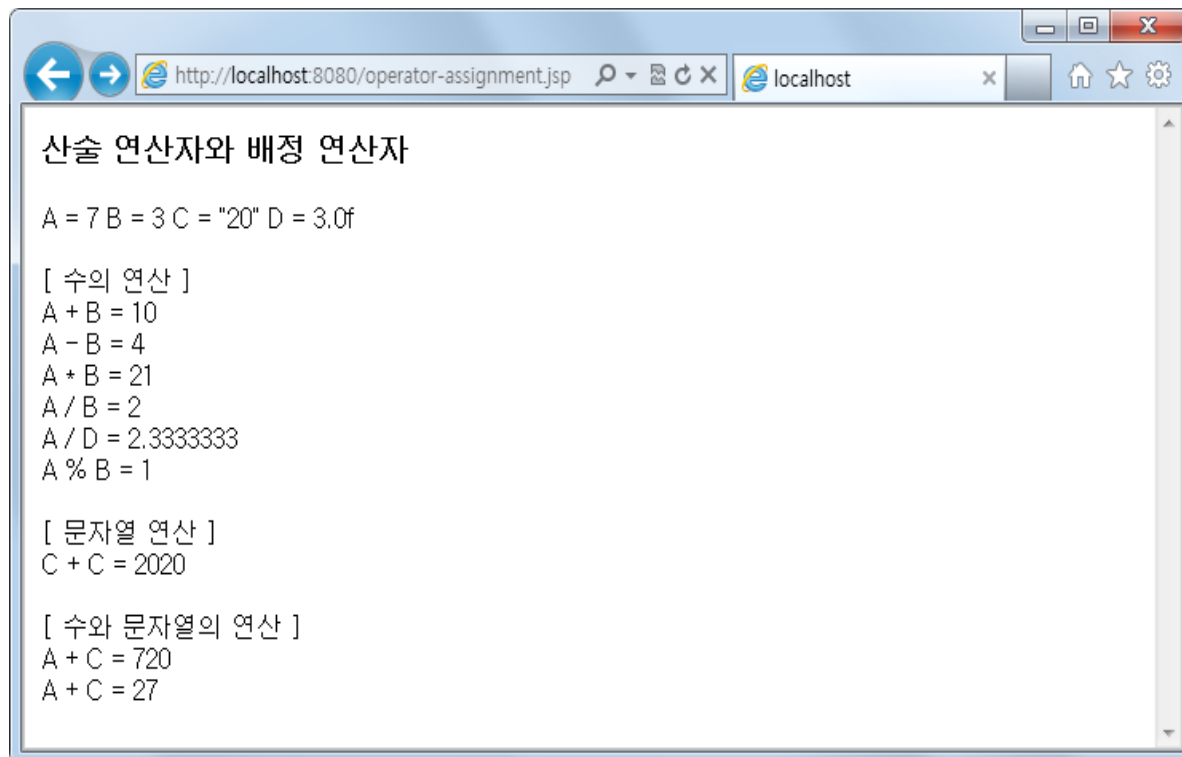
■ 예제 : operator-assignment.jsp

```
2: <h3> 산술 연산자와 배정 연산자 </h3>
3: <%
4:     int a = 7;
5:     int b = 3;
6:     String c = "20";
7:     float d = 3.0f;
8: %>
9:   A = 7   B = 3   C = "20"   D = 3.0f <br><br>
10:  [ 수의 연산 ] <br>
11:   A + B = <%= a + b %> <br>
12:   A - B = <%= a - b %> <br>
13:   A * B = <%= a * b %> <br>
14:   A / B = <%= a / b %> <br>
15:   A / D = <%= a / d %> <br>
16:   A % B = <%= a % b %> <br><br>
17:  [ 문자열 연산 ] <br>
18:   C + C = <%= c + c %> <br><br>
19:  [ 수와 문자열의 연산 ] <br>
20:   A + C = <%= a + c %> <br>
21:   A + C = <%= a + Integer.parseInt(c) %>
```

6.3.3 연산자

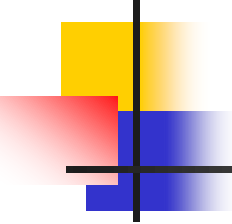
교재 p.572

■ 실행 결과



The screenshot shows a web browser window with the address bar displaying `http://localhost:8080/operator-assignment.jsp`. The page content is as follows:

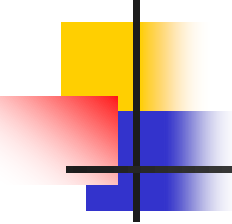
```
산술 연산자와 배정 연산자  
  
A = 7 B = 3 C = "20" D = 3.0f  
  
[ 수의 연산 ]  
A + B = 10  
A - B = 4  
A * B = 21  
A / B = 2  
A / D = 2.3333333  
A % B = 1  
  
[ 문자열 연산 ]  
C + C = 2020  
  
[ 수와 문자열의 연산 ]  
A + C = 720  
A + C = 27
```



6.3.4 문장

교재 p.572

- 조건문
- 반복문



6.3.4.1 조건문

교재 p.573

■ if 문

사용 예 :

(a) if 문

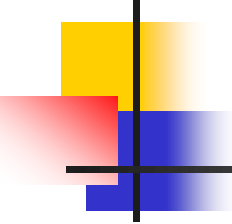
```
if(조건식1)
{
    문장_리스트1
}
```

(b) if-else 문

```
if(조건식1)
{
    문장_리스트1
}
else
{
    문장_리스트2
}
```

(c) if-else if-else 문

```
if(조건식1)
{
    문장_리스트1
}
else if(조건식2)
{
    문장_리스트2
}
else
{
    문장_리스트3
}
```



6.3.4.1 조건문

교재 p.574

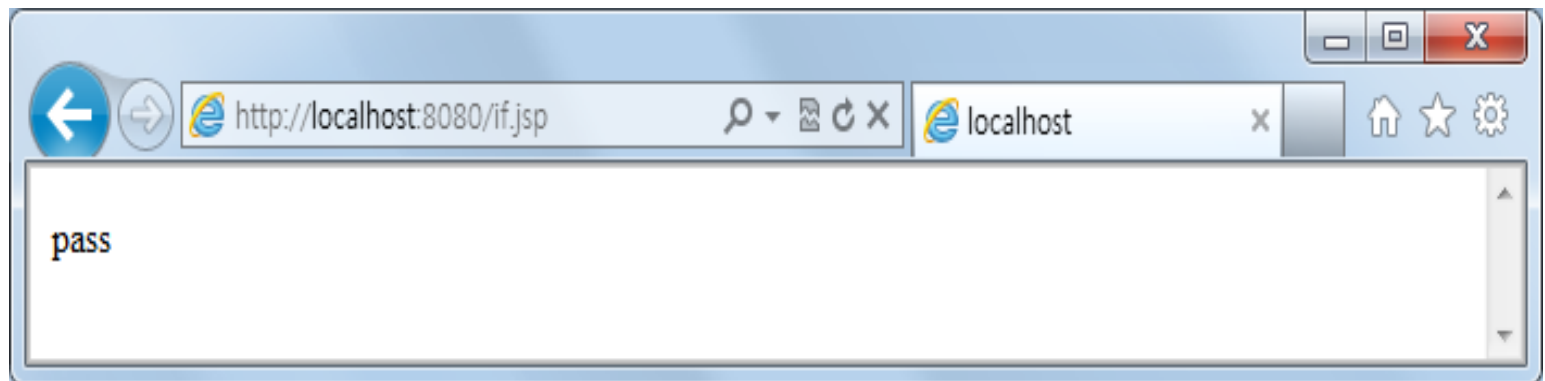
■ 예제 : if.jsp

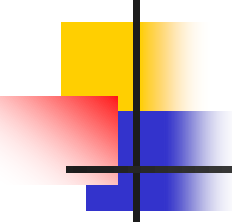
```
1: <%  
2:     int score;  
3:     score = 95;  
4:     String temp;  
5:  
6:     if(score >= 90)  
7:         temp = "pass";  
8:     else  
9:         temp = "fail";  
10: %>  
11: <%= temp %>
```

6.3.4.1 조건문

교재 p.574

■ 실행 결과





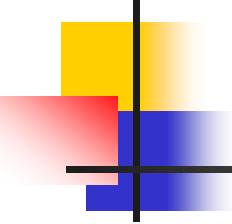
6.3.4.1 조건문

교재 p.574

■ switch 문

형 식 :

```
switch (식)
{
    case 값1 : 문장_리스트1
                break;
    case 값2 : 문장_리스트2
                break;
    ...
    case 값n : 문장_리스트n
                break;
    [ default : 문장_리스트n+1 ]
}
```



6.3.4.1 조건문

교재 p.575

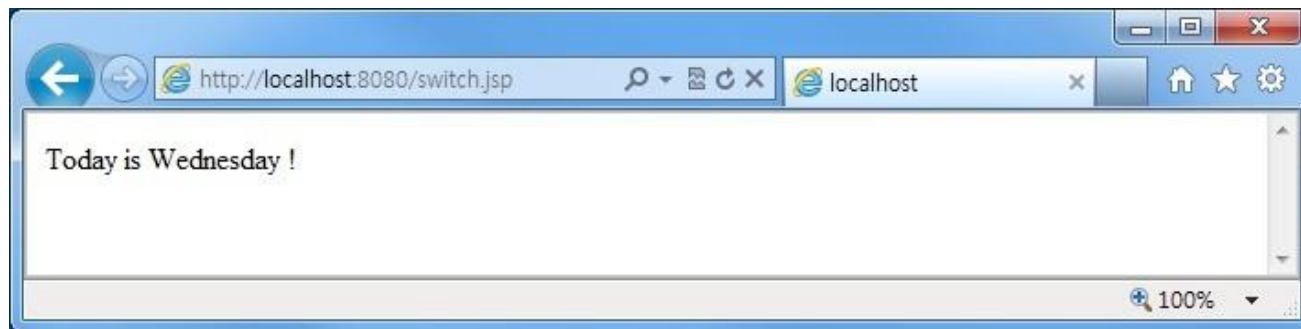
■ 예제 : switch.jsp

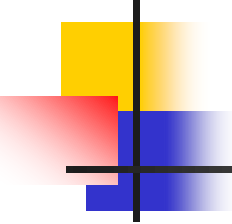
```
1:  <%
2:      int today = 3;
3:      String day;
4:
5:      switch ( today ){
6:          case 1 :
7:              day = "Monday"; break;
8:          case 2 :
9:              day = "Tuesday"; break;
10:         case 3 :
11:             day = "Wednesday"; break;
12:         case 4 :
13:             day = "Thursday"; break;
14:         case 5 :
15:             day = "Friday"; break;
16:         case 6 :
17:             day = "Saturday"; break;
18:         default :
19:             day = "Sunday";
20:     }
21:  %>
22:  Today is <%= day%> !
```

6.3.4.1 조건문

교재 p.576

- 실행 결과





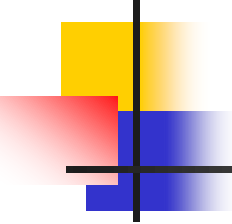
6.3.4.2 반복문

교재 p.576

■ for 문

형 식 :

```
for(초기화식; 조건식; 증감식) {  
    문장_리스트  
}
```



6.3.4.2 반복문

교재 p.577

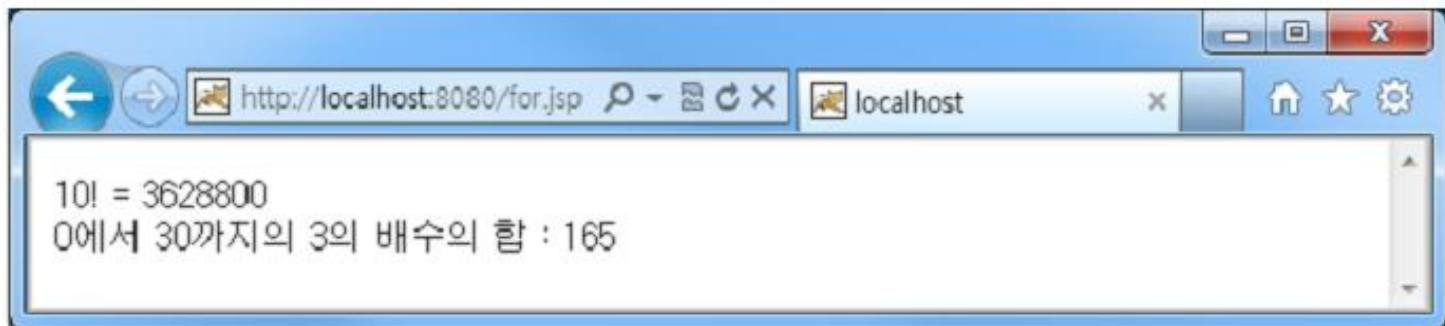
■ 예제 : for.jsp

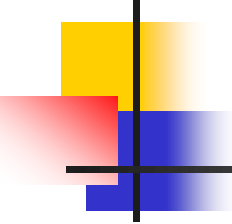
```
2 : <%
3 :     int i, fac;
4 :     fac = 1;
5 :     for(i = 1; i <= 10; i++){
6 :         fac = fac * i;
7 :     }
8 : %>
9 : 10! = <%= fac %> <br>
10 : <%
11 :     int temp = 0;
12 :     for(i = 0; i <= 30; i = i + 3){
13 :         temp = temp + i;
14 :     }
15 : %>
16 : 0에서 30까지의 3의 배수의 합 : <%= temp %>
```

6.3.4.2 반복문

교재 p.577

- 실행 결과





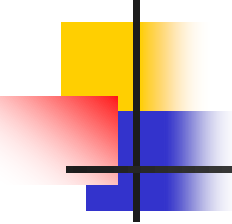
6.3.4.2 반복문

교재 p.578

■ while 문

형 식 :

```
while (조건식) {  
    문장_리스트  
    증감식;  
}
```



6.3.4.2 반복문

교재 p.578

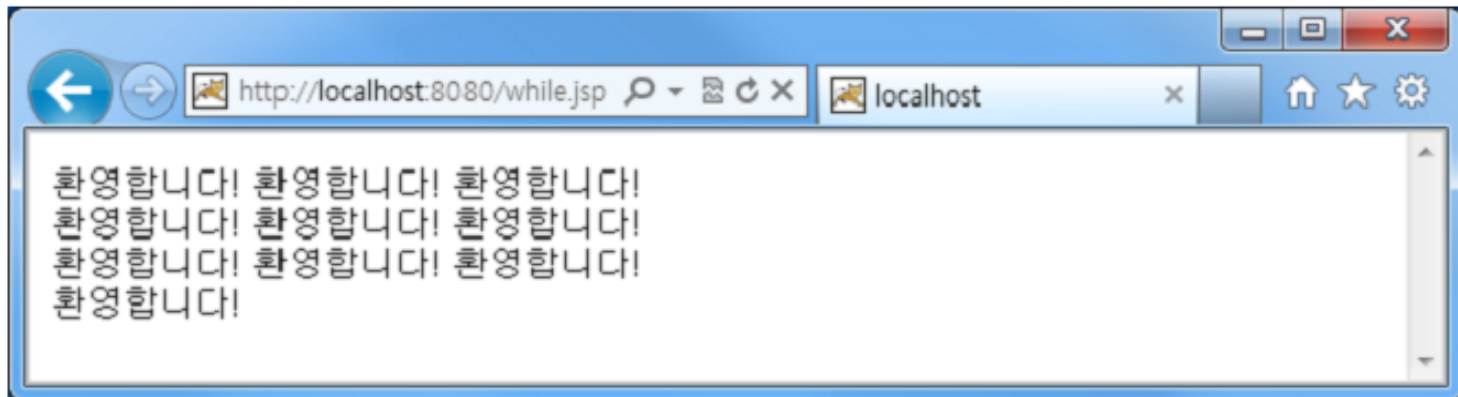
■ 예제 : while.jsp

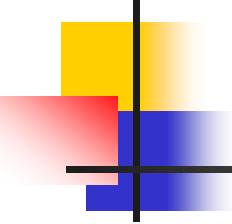
```
2 : <%
3 :     int i = 0;
4 :     while( i < 10 ) {
5 :         i = i + 1;
6 :     %>
7 :
8 :         환영합니다!
9 :
10 : <%     if(( i % 3 ) == 0){ %>
11 :
12 :         <br>
13 :
14 : <%     }
15 :     } %>
```


6.3.4.2 반복문

교재 p.579

- 실행 결과





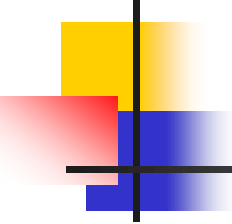
6.3.4.2 반복문

교재 p.579

- do while 문

형 식 :

```
do {   문장_리스트  
      증감식;  
} while (조건식);
```



6.3.4.2 반복문

교재 p.580

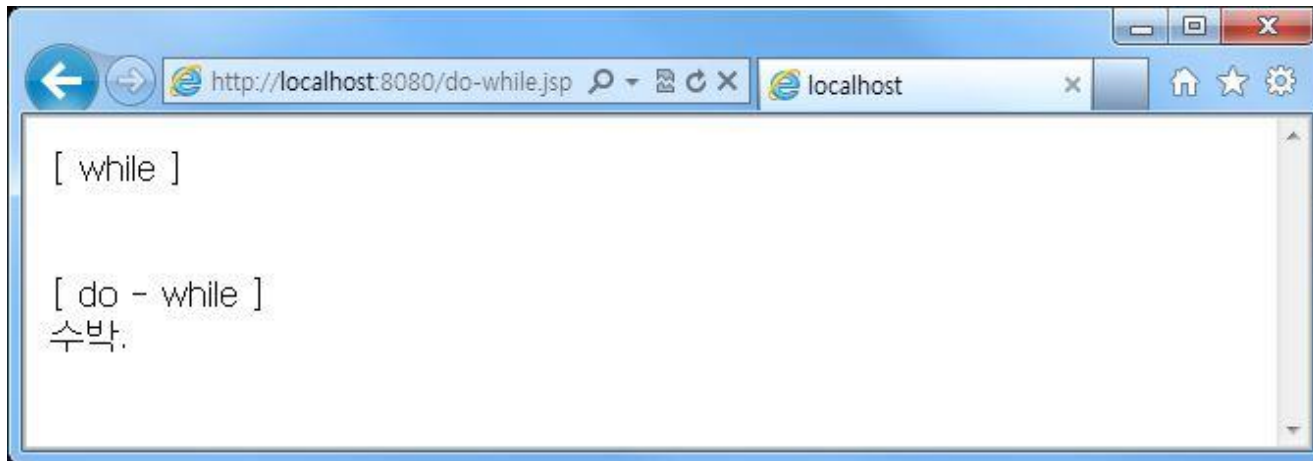
■ 예제 : do-while.jsp

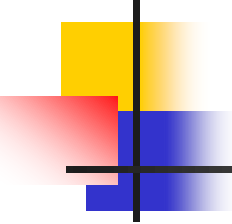
```
2 : [ while ] <br><br>
3 : <%
4 :     int i;
5 :     i = 0;
6 :     while(i < 0) {
7 :         i = i + 1;
8 :         out.println("수박.");
9 :     }
10 : %>
11 :
12 : <br> [ do - while ] <br>
13 : <%
14 :     i = 0;
15 :     do {
16 :         i = i + 1;
17 :         out.println("수박.");
18 :     } while(i < 0);
19 : %>
```

6.3.4.2 반복문

교재 p.580

■ 실행 결과

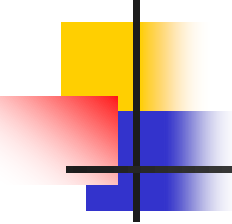




6.3.5 클래스

교재 p.581

- 자바에서 정의된 클래스의 메소드들을 사용할 수 있음
- Integer 클래스, String 클래스, Date 클래스, Math 클래스, File 클래스의 주요 메소드들을 살펴봄



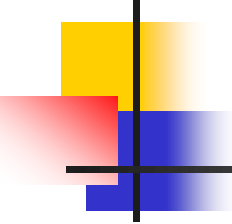
6.3.5 클래스

교재 p.581

- Integer 클래스
 - 정수 처리

[표 6.8] Integer 클래스의 주요 메소드

메소드	설명
toString()	데이터를 문자열로 변환
parseInt("문자열")	문자열 데이터를 정수형으로 변환
valueOf("문자열")	문자열 데이터를 정수형 객체로 반환
equals(객체)	객체의 정수값이 같으면 true, 아니면 false 반환



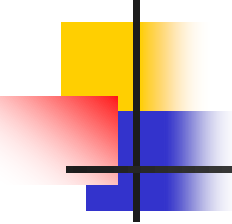
6.3.5 클래스

교재 p.581

- String 클래스
 - 문자열 다룸

[표 6.9] String 클래스의 주요 메소드

메소드	설명
toUpperCase()	알파벳 글자를 대문자로 변환
toLowerCase()	알파벳 글자를 소문자로 변환
equals(객체)	지정한 객체의 두 문자열이 같으면 true, 아니면 false 반환
concat("문자열")	문자열 연결
substring(인덱스1, 인덱스2)	인덱스1에서 인덱스2 전까지 문자열 반환, 인덱스2 생략되면 끝까지 문자열 반환
replace(기존 문자, 새로운 문자)	기존 문자를 새로운 문자로 대체
trim()	공백 제거



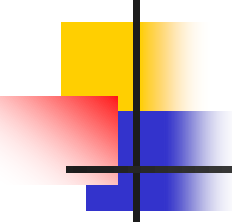
6.3.5 클래스

교재 p.582

- Date 클래스
 - 날짜와 시각 처리

[표 6.10] Date 클래스의 주요 메소드

메소드	설명
getDate()	날짜
getYear()	연
getMonth()	월
getDay()	일
getHours()	시
getMinutes()	분
toLocaleString()	현재 지역의 날짜



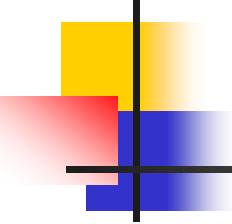
6.3.5 클래스

교재 p.582

- Math 클래스
 - 수학 계산

[표 6.11] Math 클래스의 주요 메소드

메소드	설명
abs(n)	n의 절대값
sqrt(n)	n의 제곱근
sin(n)	n의 사인값
cos(n)	n의 코사인값
tan(n)	n의 탄젠트값
max(n1, n2)	n1과 n2 중의 최대값
min(n1, n2)	n1과 n2 중의 최소값



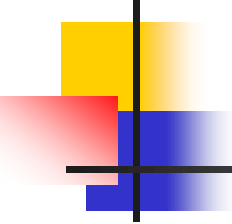
6.3.5 클래스

교재 p.583

- File 클래스
 - 파일이나 디렉토리 처리

[표 6.12] File 클래스의 주요 메소드

메소드	설명
canRead()	파일이 읽기가 가능하면 true, 아니면 false
canWrite()	파일이 쓰기가 가능하면 true, 아니면 false
delete()	파일 삭제
getPath()	파일 경로 반환
getName()	파일 이름 반환
length()	파일 길이 반환
mkdir()	디렉토리 생성

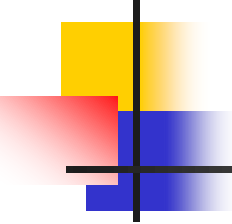


6.3.5 클래스

교재 p.583

■ 예제 : class-method.jsp

```
4 : <b> Integer 클래스의 메소드 </b> <hr>
5 : <%
6 :     out.println("Integer.toString(56) => (string)" + Integer.toString(56) + "<br>");
7 :     out.println("Integer.parseInt(W\"56W") => (int)" + Integer.parseInt("56") + "<br>");
8 : %>
9 : <br>
10 : <b> String 클래스의 메소드 </b> <hr>
11 : <%
12 :     String str = "milk";
13 :     String str2 = "MILK";
14 : %>
15 : <%= "sting : milk => 대문자 : " + str.toUpperCase()+" => 소문자 : " +
      str2.toLowerCase() %>
16 : <br> <br>
17 : <%
18 :     out.println("HOUSE".equals("House"));
19 : %>
```



6.3.5 클래스

교재 p.583

■ 예제 : class-method.jsp(계속)

```
20 : <br><br>
21 : <b> Date 클래스의 메소드 </b><hr>
22 : <%!
23 :     Date d = new Date();
24 : %>
25 : <%= d.getYear() + 1900 %>-<%= d.getMonth() + 1 %>-<%= d.getDate() %>
26 : <br><br>
27 : <b> Math 클래스의 메소드</b><hr>
28 : <%= "-5의 절대값 : " + Math.abs(-5) + "<br> 4의 제곱근 : " + Math.sqrt(4)%>
```

6.3.5 클래스

교재 p.584

■ 실행 결과



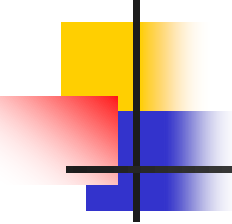
The screenshot shows a web browser window with the address bar displaying `http://localhost:8080/class-method.jsp`. The page content is organized into sections for different Java classes, each with a title and a horizontal line separator. The sections are: **Integer 클래스의 메소드** (Integer class methods) showing `Integer.toString(56) => (string)56` and `Integer.parseInt("56") => (int)56`; **String 클래스의 메소드** (String class methods) showing `sting : milk => 대문자 : MILK => 소문자 : milk` and `false`; **Date 클래스의 메소드** (Date class methods) showing `2012-12-7`; and **Math 클래스의 메소드** (Math class methods) showing `-5의 절댓값 : 5` and `4의 제곱근 : 2.0`.

```
Integer 클래스의 메소드
Integer.toString(56) => (string)56
Integer.parseInt("56") => (int)56

String 클래스의 메소드
sting : milk => 대문자 : MILK => 소문자 : milk
false

Date 클래스의 메소드
2012-12-7

Math 클래스의 메소드
-5의 절댓값 : 5
4의 제곱근 : 2.0
```



6.3.6 함수

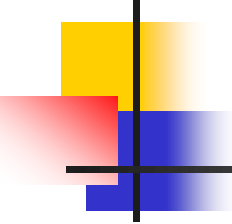
교재 p.584

- 사용자가 특정 기능을 수행하는 부분을 함수로 정의하여 사용함
- 함수 선언

```
형식 :  
<%  
    [리턴_타입] 함수명( [매개변수_리스트] ) {  
        문장_리스트  
        [ return 결과값; ]  
    }  
%>
```

- 함수 호출

```
형식 :  
<%  
    함수명( [매개변수_리스트] );  
%>
```

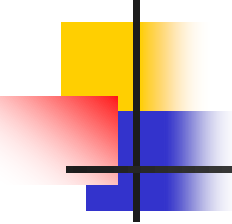


6.3.6 함수

교재 p.585

- 예제 : function.jsp
 - 함수 선언

```
2 :  <%!  
3 :      int p, q;  
4 :      int square(int i,int j){  
5 :          int value = 1;  
6 :          int k = 0;  
7 :          while (k < j){  
8 :              value = value * i;  
9 :              k = k + 1;  
10 :          }  
11 :          return value;  
12 :      }  
13 : %>
```



6.3.6 함수

교재 p.585

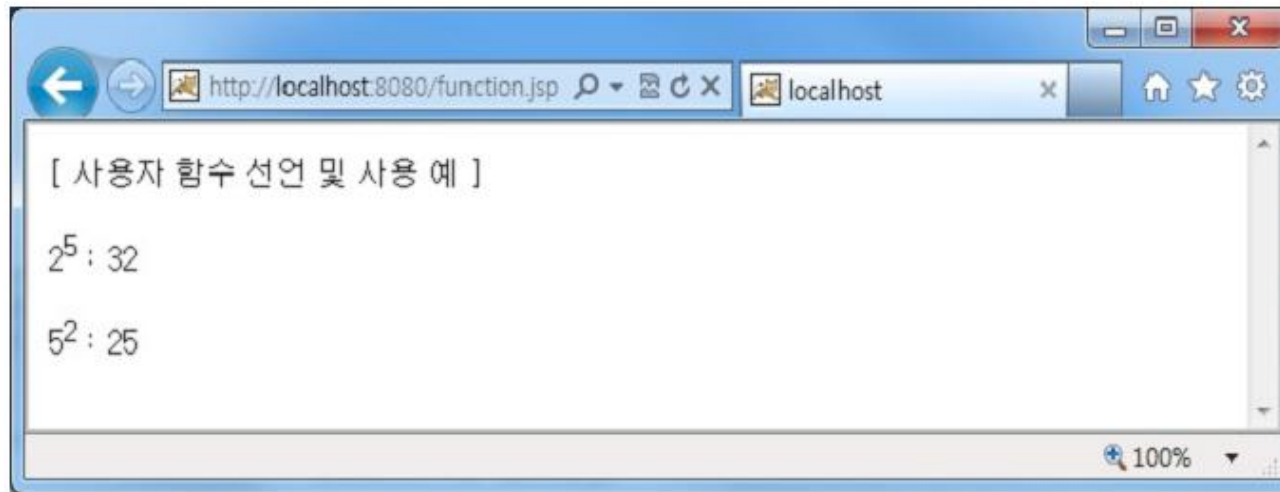
- 예제 : function.jsp(계속)
 - 함수 호출

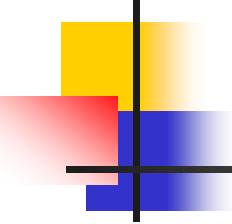
```
14 :  
15 : <%  
16 :             out.println("[ 사용자 함수 선언 및 사용 예 ]");  
17 :             p = square(2, 5);  
18 : %>  
19 :  
20 : <br><br> 2<sup>5</sup> : <%= p %>  
21 : <%  
22 :             q = square(5, 2);  
23 : %>  
24 : <br><br> 5<sup>2</sup> : <%= q %>
```


6.3.6 함수

교재 p.586

■ 실행 결과



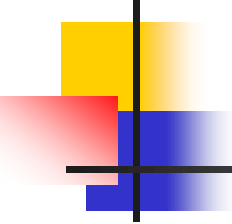


6.3.7 예외 처리

교재 p.587

■ try-catch-finally 문

```
형 식 : try {  
        문장_리스트  
    }  
    catch(예외타입1, 식별자1) {  
        문장_리스트  
    }  
    [ catch(예외타입2, 식별자2) {  
        문장_리스트  
    } ]*  
    [ finally {  
        예외 발생과 관계없이 무조건 수행되는 코드  
    } ]
```



6.3.7 예외 처리

교재 p.588

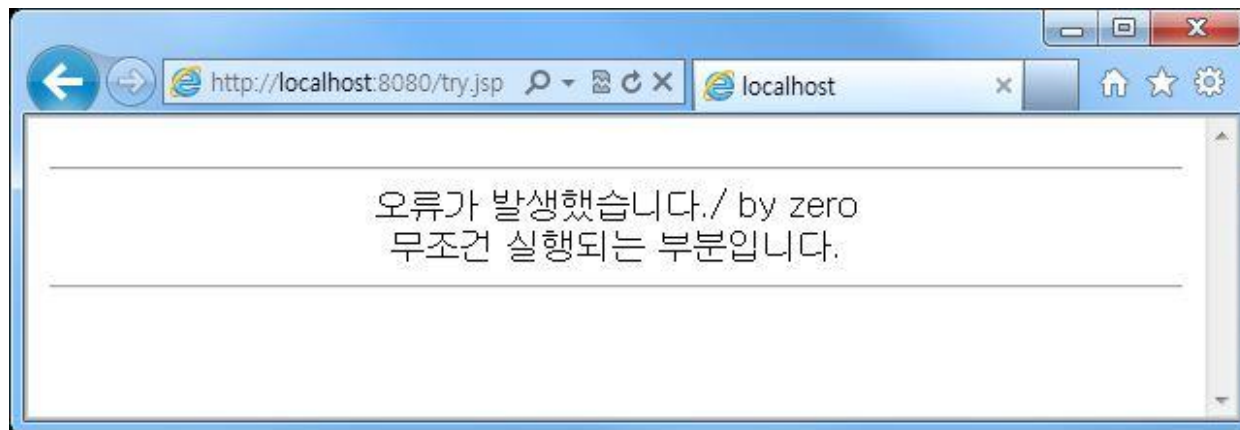
■ 예제 : try.jsp

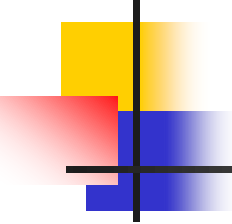
```
6 : <%
7 :     try{
8 :         int a = 0;
9 :         int num = 100 / a;
10 :
11 :         out.println(num);
12 :     }
13 :     catch(Exception e){
14 :         out.println("오류가 발생했습니다." + e.getMessage() + "<br>");
15 :     }
16 :     finally{
17 :         out.println("무조건 실행되는 부분입니다.");
18 :     }
19 : %>
```

6.3.7 예외 처리

교재 p.588

- 실행 결과

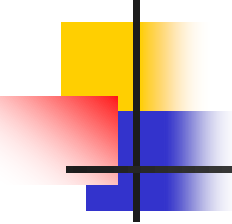




6.4 JSP 내장 객체

교재 p.589

- request 객체
- response 객체
- out 객체
- session 객체
- application 객체



6.4.1 request 객체

교재 p.589

- 서버에서 클라이언트에게 데이터를 요청할 때 사용하는 객체

형 식 :

```
request.메소드( [변수] );
```

[표 6.13] request 객체의 주요 메소드

메소드	내용
String getParameter(name)	매개변수 name의 값을 반환하며, 지정된 매개변수가 없으면 null 반환
String getCharacterEncoding()	전송된 데이터의 문자 인코딩 방식 반환

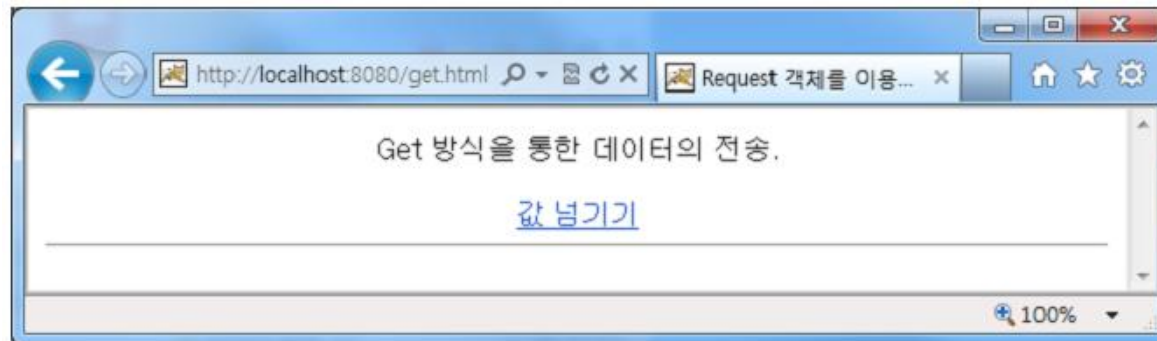
6.4.1 request 객체

교재 p.590

■ 예제 : get.html

```
5:   Get 방식을 통한 데이터의 전송.<p>
6:   <a href="get.jsp?name=강기영&id=20132292&dept=컴퓨터공학과">
7:       값 넘기기
8:   </a><br>
```

■ 실행 결과



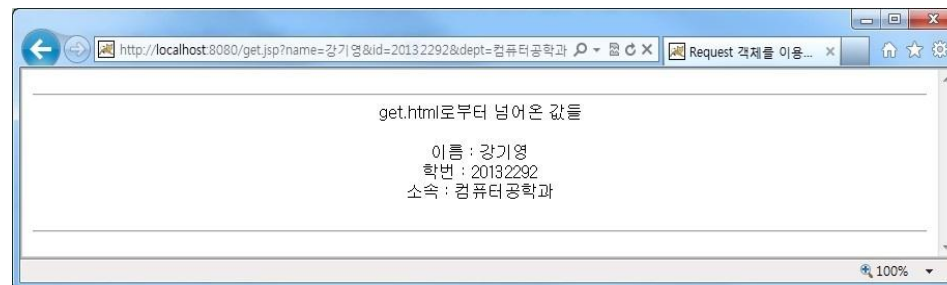
6.4.1 request 객체

교재 p.591

■ 예제 : get.jsp

```
8: <%
9:     String name = request.getParameter("name");
10:     name = new String(name.getBytes("ISO-8859-1"), "euc-kr");
11:     String id = request.getParameter("id");
12:     String dept = request.getParameter("dept");
13:     dept = new String(dept.getBytes("ISO-8859-1"), "euc-kr");
14:
15:     out.println("이름 : " + name + "<br>");
16:     out.println("학번 : " + id + "<br>");
17:     out.println("소속 : " + dept + "<br>");
18: %> <br> <br>
```

■ 실행 결과



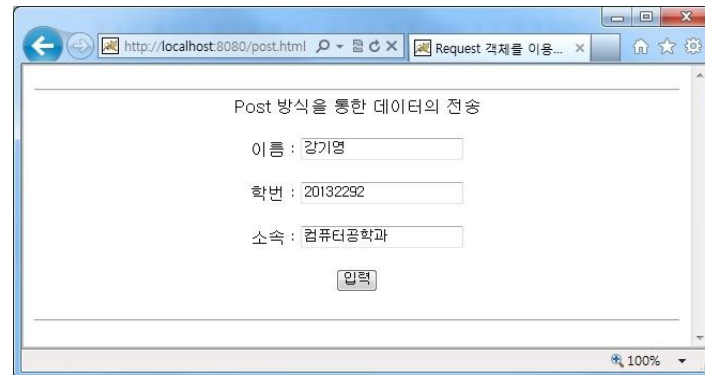
6.4.1 request 객체

교재 p.592

■ 예제 : post.html

```
7: <form action="post.jsp" method="post">
8:     이름 : <input type="text" name="name"> <br> <br>
9:     학번 : <input type="text" name="id"> <br> <br>
10:    소속 : <input type="text" name="dept"> <br> <br>
11:    <input type="submit" value="입력">
12: </form>
```

■ 실행 결과



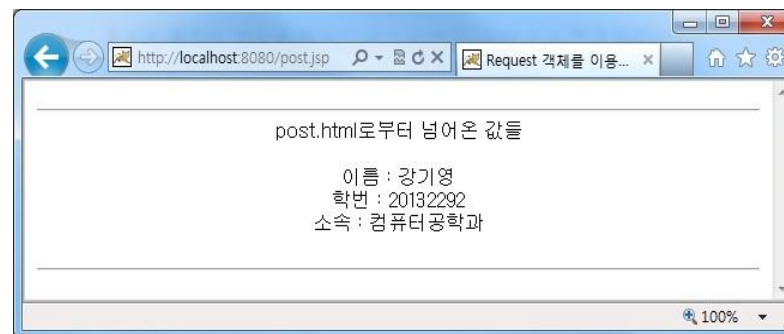
6.4.1 request 객체

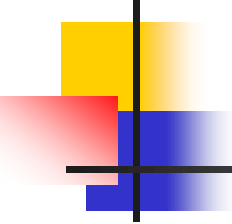
교재 p.593

■ 예제 : post.jsp

```
9:  <%  
10:      String name = request.getParameter("name");  
11:      String id = request.getParameter("id");  
12:      String dept = request.getParameter("dept");  
13:  
14:      out.println("이름 : " + name + "<br>");  
15:      out.println("학번 : " + id + "<br>");  
16:      out.println("소속 : " + dept + "<br>");  
17:  %>
```

■ 실행 결과





6.4.2 response 객체

교재 p.594

- 서버에서 클라이언트로 데이터를 응답할 때 사용

형 식 :

```
response.메소드( [변수] );
```

[표 6.14] response 객체의 주요 메소드

메소드	내용
sendRedirect("url")	url의 페이지로 이동

6.4.2 response 객체

교재 p.595

■ 예제 : response-login.html

```
9 : <form action="response-loginresult.jsp" method="post">
10 :     이름 : <input type="text" name="name" size="40"> <br>
11 :     비밀번호 : <input type="password" name="pwd" size="40"> <br> <br>
12 :     <input type="submit" value="로그인">
13 :     <input type="reset" value="원래대로">
14 : </form>
```

■ 실행 결과

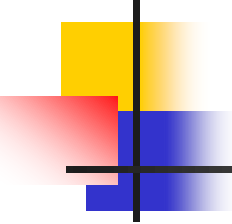
Response.sendRedirect 예제

이름과 암호를 입력하십시오.

이름 : admin

비밀번호 :

로그인 원래대로



6.4.2 response 객체

교재 p.596

- 예제 : response-loginresult.jsp
 - 사용자로부터 입력받은 이름과 비밀번호 처리

```
5 :    <%  
6 :        String name = request.getParameter("name");  
7 :        String pass = request.getParameter("pwd");  
8 :  
9 :        if(name.equals("admin") && pass.equals("1234")){  
10 :            response.sendRedirect("response-success.html");  
11 :        } else {  
12 :            response.sendRedirect("response-fail.html");  
13 :        }  
14 :  
15 :    %>
```

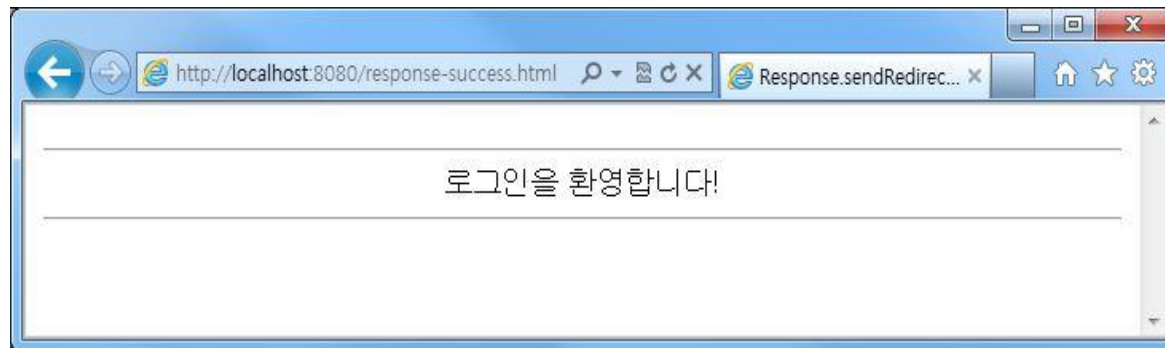
6.4.2 response 객체

교재 p.596

- 예제 : response-success.html

```
4 :      <hr><center>
5 :          로그인을 환영합니다! <br>
6 :      </center><hr>
```

- 실행 결과



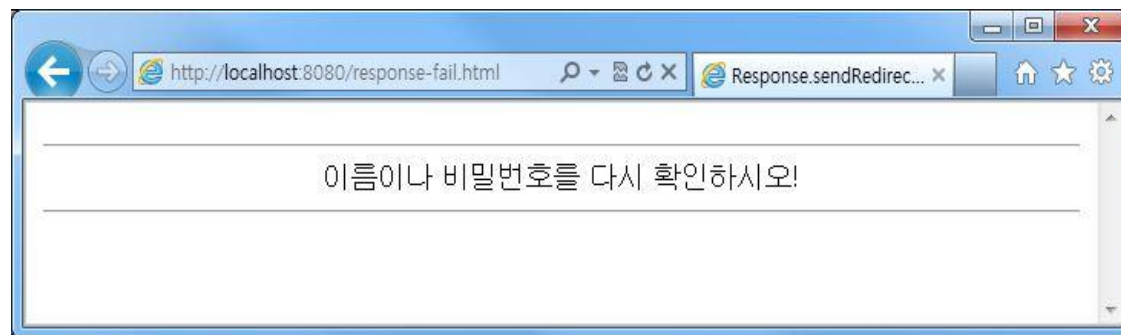
6.4.2 response 객체

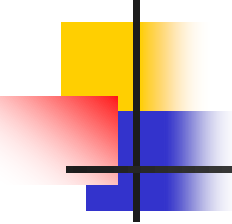
교재 p.597

- 예제 : response-fail.html

```
5 :      <hr><center>
6 :      이름이나 비밀번호를 다시 확인하십시오! <br>
7 :      </center><hr>
```

- 실행 결과





6.4.3 out 객체

교재 p.597

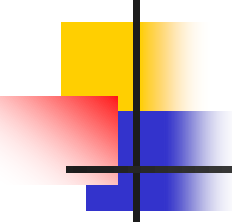
- 문자열이나 변수에 저장된 값을 출력할 때 사용

형 식 :

```
out.메소드( [변수] );
```

[표 6.15] out 객체의 주요 메소드

메소드	내용
print("문자열")	데이터 출력
println("문자열")	데이터 출력 후, \r\n 또는 \n 출력 (줄바꿈)
newline()	\r\n 또는 \n 출력 (줄바꿈)



6.4.3 out 객체

교재 p.598

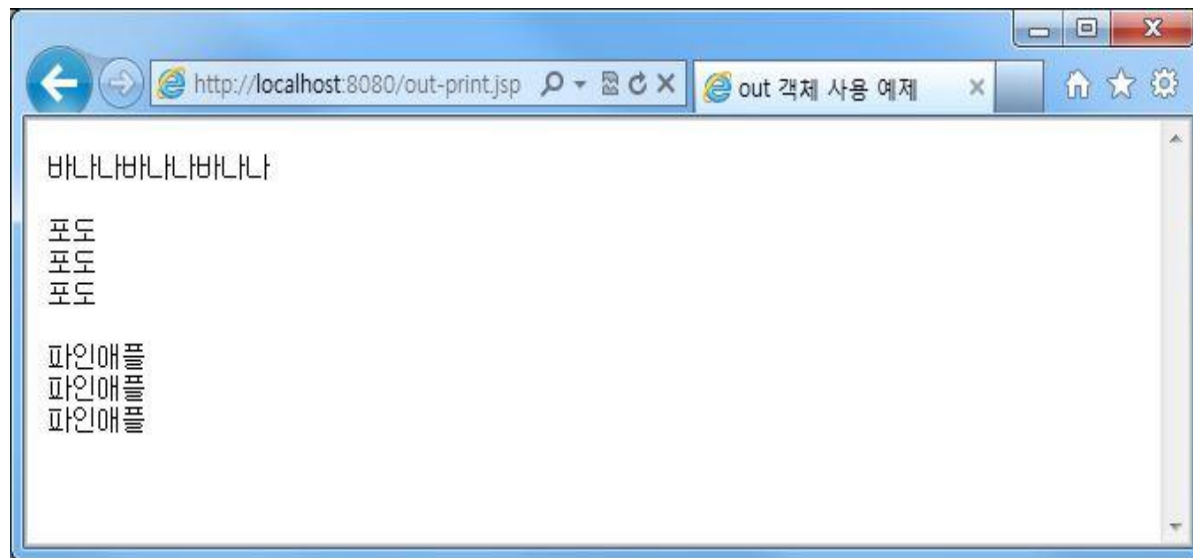
■ 예제 : out-print.jsp

```
5 : <pre>
6 : <%
7 :     for(int i = 0; i < 3; i++) {
8 :         out.print("바나나");
9 :     }
10 : %>
11 :
12 : <%
13 :     for(int i = 0; i < 3; i++){
14 :         out.println("포도");
15 :     }
16 : %>
17 : <%
18 :     for(int i = 0; i < 3; i++){
19 :         out.print("파인애플");
20 :         out.newLine();
21 :     }
22 : %>
23 : </pre>
```

6.4.3 out 객체

교재 p.599

■ 실행 결과



6.4.4 session 객체

교재 p.599

- 사용자의 세션 정보를 서버와의 세션이 유지되는 동안 보관하는 객체

형 식 :

```
session.메소드( [변수] );
```

[표 6.16] session 객체의 주요 메소드

메소드	내용
getId()	ID 반환
getCreationTime()	생성 시각 반환
getLastAccessedTime()	마지막 접근 시각 반환
getAttribute()	저장된 속성의 값 반환
getAttributeNames()	저장된 속성 이름 리스트 반환
setAttribute("문자열", 객체)	속성의 값 설정
removeAttribute("문자열")	저장된 속성 삭제

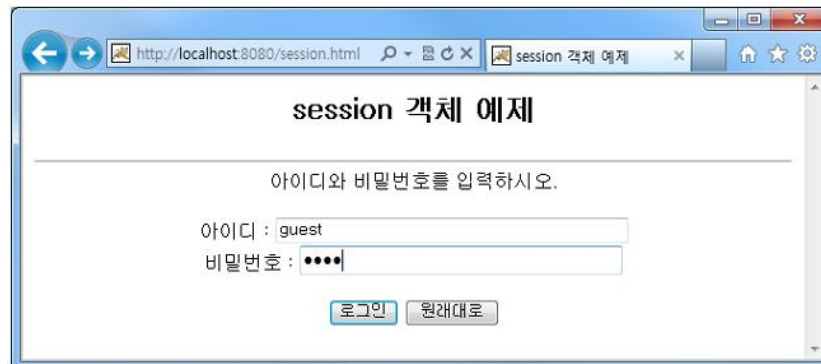
6.4.4 session 객체

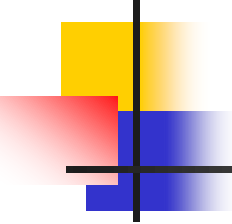
교재 p.600

■ 예제 : session.html

```
9 : <form action="session-set.jsp" method="post">
10 :     아이디 : <input type="text" name="id" size="40"> <br>
11 :     비밀번호 : <input type="password" name="pw" size="40"> <p>
12 :     <input type="submit" value="로그인">
13 :     <input type="reset" value="원래대로">
14 : </form>
```

■ 실행 결과





6.4.4 session 객체

교재 p.601

■ 예제 : session-set.jsp

```
7 :  <%
8 :      String id = request.getParameter("id");
9 :      String pw = request.getParameter("pw");
10 :  %>
11 :
12 :      아이디는 <%= id %>입니다. <br>
13 :      비밀번호는 <%= pw %>입니다. <br>
14 :
15 :  <%
16 :      session.setAttribute("id", id);
17 :      session.setAttribute("pwd", pw);
18 :  %>
19 :
20 :  <a href="session-get.jsp"> session-get 페이지로 이동 </a>
```

6.4.4 session 객체

교재 p.602

- 실행 결과



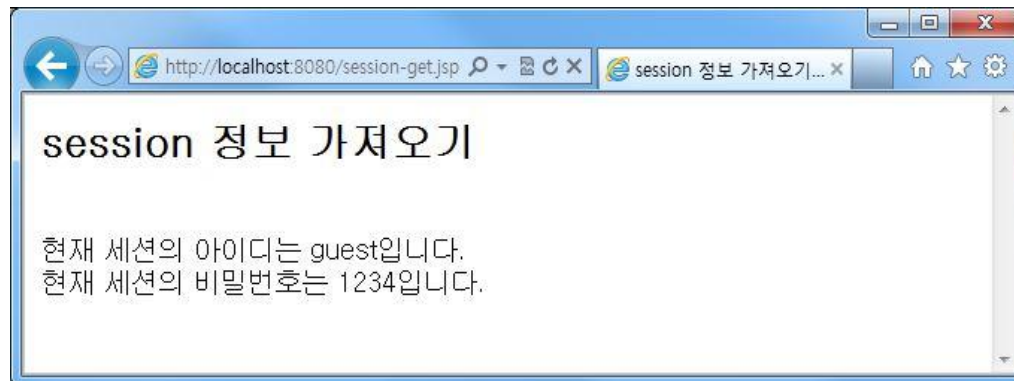
6.4.4 session 객체

교재 p.602

■ 예제 : session-get.jsp

```
6 : <h2> session 정보 가져오기 </h2>
7 : <br>
8 :     현재 세션의 아이디는 <%= session.getAttribute("id") %>입니다.<br>
9 :     현재 세션의 비밀번호는 <%= session.getAttribute("pwd") %>입니다.<br>
```

■ 실행 결과



6.4.5 application 객체

교재 p.603

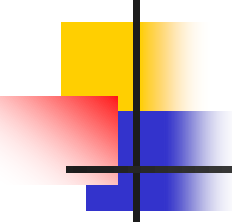
- 웹 애플리케이션의 페이지들이 정보 공유

형 식 :

```
application.메소드( [변수] );
```

[표 6.17] application 객체의 주요 메소드

메소드	내용
getServerInfo()	서버 정보 반환
getAttribute("문자열")	저장된 속성의 값 반환
getAttributeNames()	저장된 속성 이름 리스트 반환
setAttribute("문자열", 객체)	속성의 값 설정
removeAttribute("문자열")	저장된 속성 삭제



6.4.5 application 객체

교재 p.604

■ 예제 : application-set.jsp

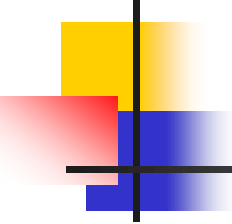
```
6 :      <%  
7 :          String value = "apple";  
8 :          application.setAttribute("fruit", value);  
9 :          String fruit = (String)application.getAttribute("fruit");  
  
10 :      %>  
11 :  
12 :      application 객체에 저장된 값 : <%= fruit%>
```

6.4.5 application 객체

교재 p.604

- 실행 결과

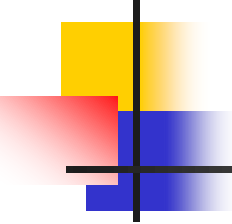




6.5 JSP와 데이터베이스 연동

교재 p.604

- JDBC를 이용한 데이터베이스 연동
- 데이터베이스 연동 예제



6.5.1 JDBC를 이용한 데이터베이스 연동

교재 p.605

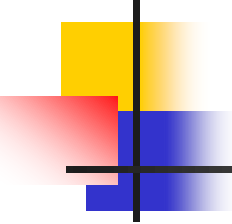
- JDBC(Java Database Connectivity)는 자바 프로그램에서 데이터베이스에 연결하기 위한 표준 인터페이스

- (1) java.sql 패키지 импорт

사용 예 : `<%@ page import = "java.sql.*" %>`

- (2) JDBC 드라이버 로드

사용 예 : `Class.forName("com.mysql.jdbc.Driver");`



6.5.1 JDBC를 이용한 데이터베이스 연동

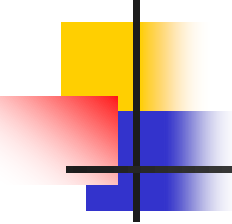
교재 p.605

- (3) 데이터베이스 경로 지정

사용 예 : `String jdbcurl = "jdbc:mysql://localhost:3306/address_db";`

- (4) Connection 객체 생성

사용 예 : `Connection conn =
DriverManager.getConnection(jdbcurl,"manager","wppass");`



6.5.1 JDBC를 이용한 데이터베이스 연동

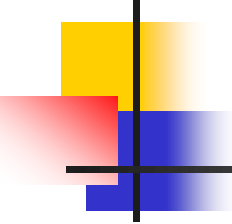
교재 p.605

- (5) Statement 객체 생성

```
사용 예 : Statement stmt = conn.createStatement();
```

- (6) SQL 문 설정

```
사용 예 : String sql1 = "select * from address_tbl";
```



6.5.1 JDBC를 이용한 데이터베이스 연동

교재 p.606

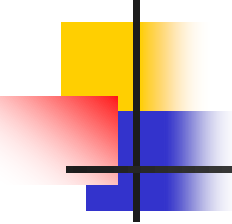
■ (7) SQL 문 실행

사용 예 : `ResultSet rs = stmt.executeQuery(sql1);`

■ (8) ResultSet 객체 사용

사용 예 :

```
while(rs.next()) {  
    <%= rs.getString("student_id")%>  
    <%= rs.getString("name")%>  
    <%= rs.getString("e_mail")%>  
    <%= rs.getString("address")%>  
}
```



6.5.1 JDBC를 이용한 데이터베이스 연동

교재 p.607

- (9) 예외 처리

```
사용 예 : out.println("DB 연동 오류입니다. : " + e.getMessage() );
```

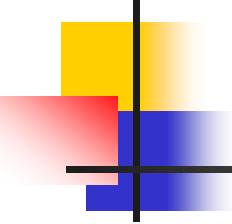

6.5.2 데이터베이스 연동 예제

교재 p.608

- 데이터베이스 연동 예제
 - 데이터베이스 이름 : address_db
 - 테이블 이름 : address_tbl

[표 6.19] address_tbl 테이블의 구성

열 이름	데이터 타입	설명
student_id	varchar	학번(기본 키)
name	varchar	이름
e_mail	varchar	전자 우편
address	varchar	주소



6.5.2 데이터베이스 연동 예제

교재 p.608

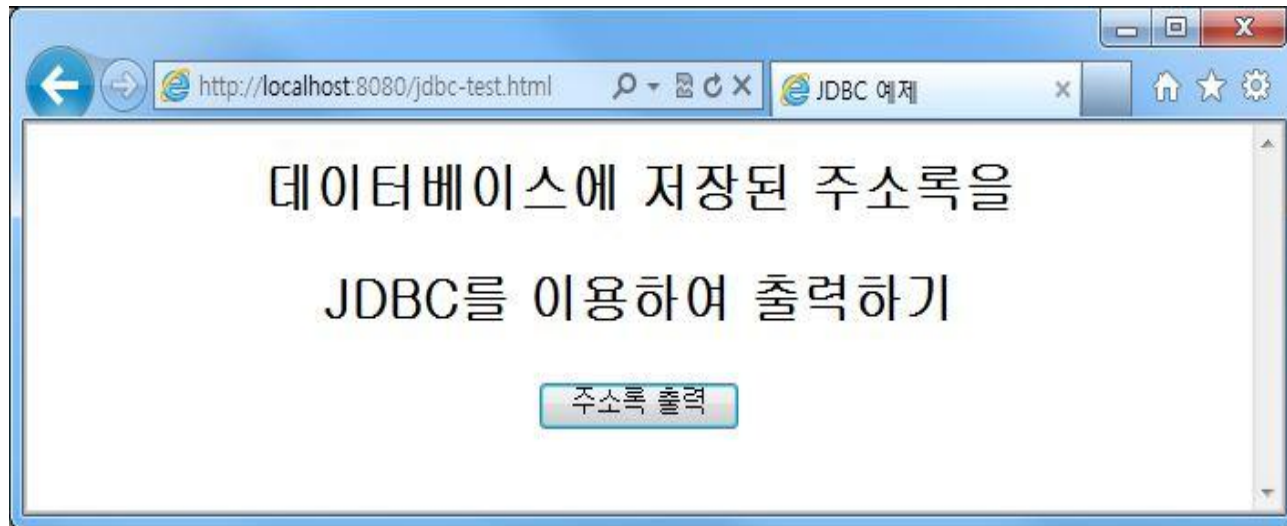
■ 예제 : jdbc-test.html

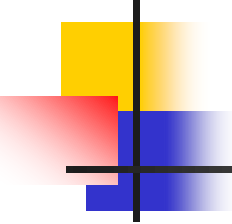
```
4 : <center>
5 :   <h2>
6 :     데이터베이스에 저장된 주소록을 <br><br> JDBC를 이용하여 출력하기
7 :   </h2>
8 :   <br><br>
9 :   <form action="jdbc-test.jsp" method="get">
10 :     <input type="submit" value="주소록 출력">
11 :   </form>
12 : </center>
```

6.5.2 데이터베이스 연동 예제

교재 p.609

- 실행 결과



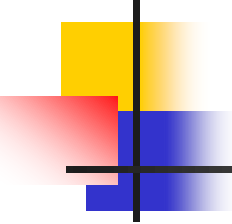


6.5.2 데이터베이스 연동 예제

교재 p.609

■ 예제 : jdbc-test.jsp

```
8 : <table border="1" align="center">
9 :     <tr>
10 :         <td align="center"> Student_ID </td>
11 :         <td align="center"> Name </td>
12 :         <td align="center"> E-mail </td>
13 :         <td align="center"> Address </td>
14 :     </tr>
15 :
16 :     <%
17 :         Connection conn = null;
18 :         Statement stmt = null;
19 :         ResultSet rs = null;
20 :
```

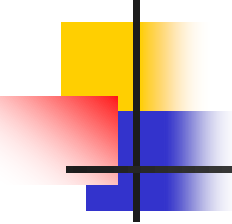


6.5.2 데이터베이스 연동 예제

교재 p.609

■ 예제 : jdbc-test.jsp(계속)

```
21 : try {
22 :     Class.forName("com.mysql.jdbc.Driver");
23 :     String jdbcurl = "jdbc:mysql://localhost:3306/address_db";
24 :     conn = DriverManager.getConnection(jdbcurl,"manager","wppass");
25 :     stmt = conn.createStatement();
26 :     String sql = "select * from address_tbl";
27 :     rs = stmt.executeQuery(sql);
28 : }
29 : catch(Exception e) {
30 :     out.println("DB 연동 오류입니다. : " + e.getMessage() );
31 : }
32 :
33 : while(rs.next()) {
34 : %>
```



6.5.2 데이터베이스 연동 예제

교재 p.610

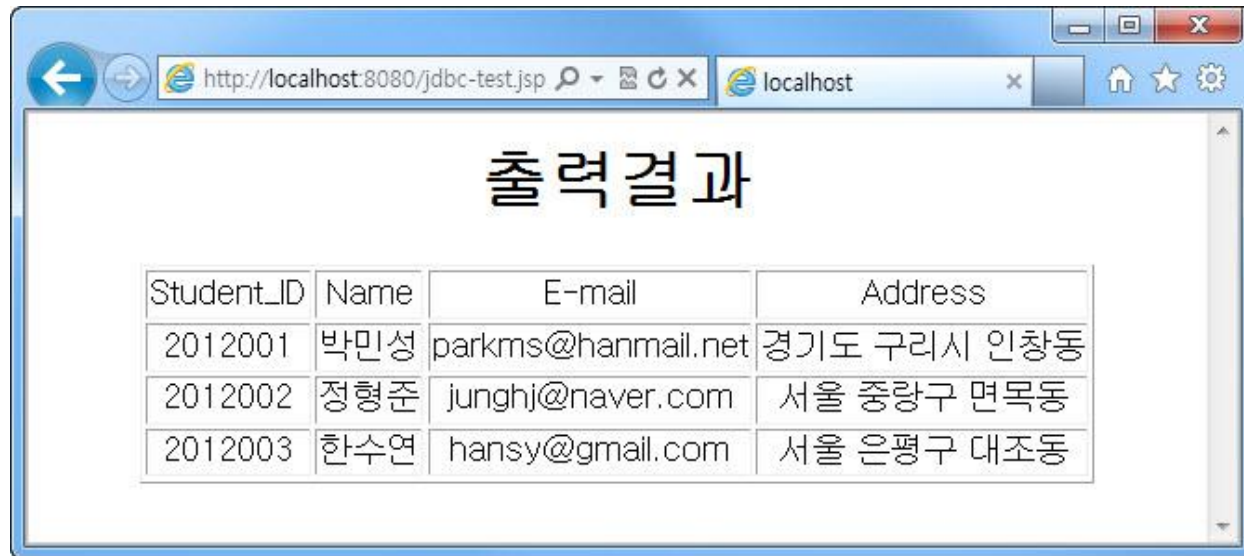
■ 예제 : jdbc-test.jsp(계속)

```
35 :      <tr>
36 :          <td align="center"> <%= rs.getString("student_id")%> </td>
37 :          <td align="center"> <%= rs.getString("name")%> </td>
38 :          <td align="center"> <%= rs.getString("e_mail")%> </td>
39 :          <td align="center"> <%= rs.getString("address")%> </td>
40 :      </tr>
41 :      <%
42 :          }
43 :      %>
44 : </table>
```

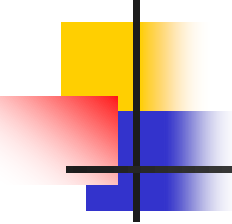
6.5.2 데이터베이스 연동 예제

교재 p.611

■ 실행 결과



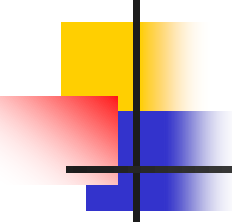
Student_ID	Name	E-mail	Address
2012001	박민성	parkms@hanmail.net	경기도 구리시 인창동
2012002	정형준	junghj@naver.com	서울 중랑구 면목동
2012003	한수연	hansy@gmail.com	서울 은평구 대조동



6.6 JSP 프로그래밍 예제

교재 p.611

- 여론조사
 - 국가나 사회의 여러 가지 문제에 대한 대중의 의견이나 경향 등에 대한 통계 조사



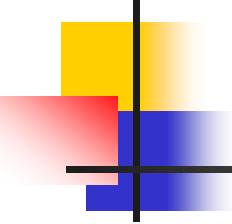
6.6.1 여론조사

교재 p.611

- 여론조사 예제
 - 데이터베이스 이름 : opinion_db
 - 테이블 이름 : opinion_tbl

[표 6.20] opinion_tbl 테이블의 구성

열 이름	데이터 타입	설명
id	int	고유번호(기본 키)
fruit	varchar	과일
sum	int	응답 수



6.6.1 여론조사

교재 p.612

- 여론조사 예제(계속)
 - 여론조사 초기 화면 : `opinion.html`
 - 의견 입력 화면 : `opinion-poll.html`
 - 여론조사 결과 화면 : `opinion-result.jsp`

6.6.1 여론조사

교재 p.612

■ 여론조사 초기 화면 : opinion.html

```
5 : <table border="0">
6 :   <tr>
7 :     <td align="center">
8 :       <h2> 여론조사 </h2>
9 :       <font size="3"> 좋아하는 과일을 조사합니다.</font> <br> <br>
10 :      <font size="3"> 투표하기 링크를 누르면 조사에 참여할 수 있습니다.</font> <br> <br>
11 :      <font size="3"> 결과보기 링크를 누르면 결과를 확인할 수 있습니다.</font> <br> <br>
12 :       <a href="opinion-poll.html"> 투표하기 </a>
13 :       <a href="opinion-result.jsp"> 결과보기 </a> <br>
14 :      
15 :    </td>
16 :  </tr>
17 : </table>
```

6.6.1 여론조사

교재 p.613

■ 실행 결과



6.6.1 여론조사

교재 p.613

■ 의견 입력 화면 : opinion-poll.html

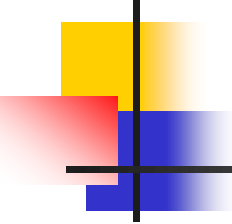
```
5 : <table border="0">
6 :   <tr><td align="center">
7 :     <h2> 여론조사 참여 </h2>
8 :     <form action="opinion-result.jsp" method="post">
9 :        좋아하는 과일을 선택해 주세요. <br><br>
10 :      <input type="radio" name="opinion_id" value="0"> 사과 <br>
11 :      <input type="radio" name="opinion_id" value="1"> 포도 <br>
12 :      <input type="radio" name="opinion_id" value="2"> 딸기 <br>
13 :      <input type="radio" name="opinion_id" value="3"> 메론 <br><br>
14 :      <input type="submit" value="투표하기">
15 :    </form>
16 :    
17 :  </td></tr>
18 :</table>
```

6.6.1 여론조사

교재 p.614

■ 실행 결과



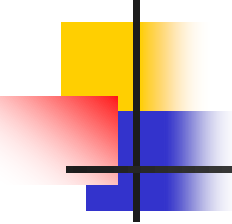


6.6.1 여론조사

교재 p.614

- 여론조사 결과 화면 : opinion-result.jsp
 - 변수 초기화

```
9 :  <%  
10 :      int choice_id, i;  
11 :      int opinion[]={0, 0, 0, 0};  
12 :      float total, rate[]={0, 0, 0, 0};  
13 :      String choice="";  
14 :      String fruit[]{"사과", "포도", "딸기", "메론"};  
15 :      String opinion_id = request.getParameter("opinion_id");  
16 :      Connection conn = null;  
17 :      Statement stmt = null;  
18 :      String sql= "";  
19 :      ResultSet rs = null;
```

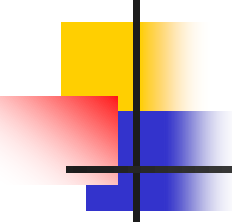


6.6.1 여론조사

교재 p.614

- 여론조사 결과 화면(계속)
 - DB 연동 부분

```
21 :    try {
22 :        Class.forName("com.mysql.jdbc.Driver");
23 :        String url = "jdbc:mysql://localhost:3306/opinion_db";
24 :        conn = DriverManager.getConnection(url,"manager","wppass");
25 :        stmt = conn.createStatement();
26 :        sql = "select * from opinion_tbl";
27 :        rs = stmt.executeQuery(sql);
28 :    }
29 :    catch(Exception e) {
30 :        out.println("DB 연동 오류입니다. : " + e.getMessage() );
31 :    }
```

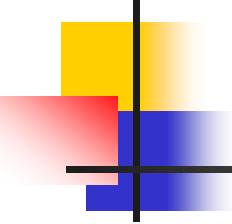



6.6.1 여론조사

교재 p.615

- 여론조사 결과 화면(계속)
 - 테이블 초기화

```
33 : if(!rs.next()) {  
34 :   for(i = 0; i < 4; i++) {  
35 :     String sql1 = "insert into opinion_tbl values (" + i + ", '" + fruit[i] + "', 0)";  
36 :     try{  
37 :       stmt.executeUpdate(sql1);  
38 :     }  
39 :     catch(Exception e) {  
40 :       out.println("DB 연동 오류입니다. : " + e.getMessage());  
41 :     }  
42 :   }  
43 : }
```

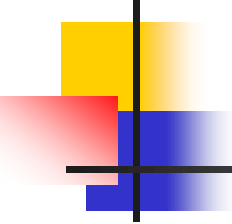


6.6.1 여론조사

교재 p.615

- 여론조사 결과 화면(계속)
 - sum을 배열 opinion에 배정하는 부분

```
44 : else {  
45 :     rs.previous();  
46 :  
47 :     i = 0;  
48 :  
49 :     while(rs.next()) {  
50 :         opinion[i] = Integer.parseInt(rs.getString("sum"));  
51 :         i++;  
52 :     }  
53 : }
```

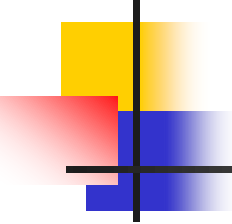


6.6.1 여론조사

교재 p.615

- 여론조사 결과 화면(계속)
 - 응답 수 증가

```
55 : if(opinion_id != null) {
56 :     String sql2 = "update opinion_tbl set sum = sum + 1 where id = " + opinion_id;
57 :     try{
58 :         stmt.executeUpdate(sql2);
59 :     }
60 :     catch(Exception e) {
61 :         out.println("DB 연동 오류입니다. : " + e.getMessage());
62 :     }
63 : }
64 :
65 : choice_id = Integer.parseInt(opinion_id);
66 : opinion[choice_id] += 1;
```



6.6.1 여론조사

교재 p.615

- 여론조사 결과 화면(계속)
 - 전체 응답자 수를 계산하는 부분
 - 각 항목의 전체에 대한 백분율을 계산하는 부분

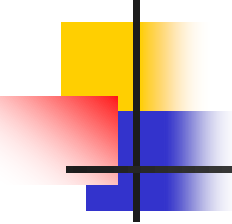
```
68 : total = opinion[0] + opinion[1] + opinion[2] + opinion[3];  
69 :  
70 : for(i = 0; i < 4; i++)  
71 :     rate[i] = (opinion[i] / total) * 100;  
72 : %>
```

6.6.1 여론조사

교재 p.616

- 여론조사 결과 화면(계속)
 - 각 항목의 백분율을 출력
 - 그만큼의 길이를 그래프로 표현

```
74 : <h2> 여론조사 결과 </h2>
75 : <table border="0" width="500">
76 :   <tr>
77 :     <td bgcolor="yellow" width="30%">
78 :       <b> 사과 : <%=Math.round(rate[0])%>% </b> </td>
79 :     <td> % height="25"> </td>
80 :   </tr>
81 :   <tr>
82 :     <td bgcolor="orange"> <b> 포도 : <%=Math.round(rate[1])%>% </b> </td>
83 :     <td> % height="25"> </td>
84 :   </tr>
85 :   . . . 생략 . . .
```



6.6.1 여론조사

교재 p.616

- 여론조사 결과 화면(계속)
 - 여론조사에 참여한 사람들의 총 인원을 나타내는 부분

```
94 : 당신은 <font color="blue"><b><%=fruit[choice_id]%></b></font>를(을) 선택하였습니다.<br>
95 : 총 <b><font color="red"><%=Math.round(total)%></font></b>명이 참여하였습니다.<br><br>
96 : <a href="opinion.html"> 처음으로 </a>
```

6.6.1 여론조사

교재 p.617

■ 실행 결과

